Assignment 9 - Requirements Document and Technical Specification Document

Technical Specification Document (version 1.0, October 29, 2026)

Team: Allison Oh, Aditya Pawar

Advisors: Jillian Aurisano, Todd Liebermann

Source: https://github.com/AllyOI/model-recommend/blob/main/Technical_Specification.md

Senior Design 1, Fall 2026

Technical Specification Document — Model Recommend

Project: Model Recommend (Concentrate AI) **Team:** Allison Oh, Aditya Pawar **Advisors:** Jillian Aurisano (Department of Computer Science), Todd Liebermann (Concentrate AI) **Document version:** 1.0 (2026-10-29) **Traces to:** *Requirements_Document.md* **v1.0 (2026-10-29)**. Every REQ, NFR and CON ID below refers to that version.

Design sources, linked rather than copied: D0 High-Level Design, D1 Detailed Design, D2 Detailed Design, Planning/Plan.md. Terms follow the glossary in the Requirements Document, Section 4.

1. Version and Change Log

Version	Date	Change	Author
1.0	2026-10-29	First version. Consolidates D0 (2026-10-02), D1 (2026-10-08) and D2 (2026-10-14) into one specification traced to Requirements Document v1.0.	Allison Oh, Aditya Pawar

Design changes since the D1 and Week 7 (D2) reviews, and why. No component was added or removed after D2. The changes below were made in D2 or are applied in this specification to close D2's known gaps.

#	Change	Why	Where
DC-01	E1 returns `202 {run_id}` and the client polls E5, instead of holding the request open up to 90 s.	US-03 needs a state between scoring and display, and hosting platforms commonly cut requests near 60 s (D2 R-01).	D2 2.2
DC-02	Cost is computed from vendor token counts × configured prices; vendors do not return cost.	Closes D2 gap G2; D0's I3 row was wrong (D2 R-04).	D2 2.4, Section 3.5
DC-03	The Server ↔ Report Store protocol is the PostgreSQL wire protocol over TLS, not "internal RPC (HTTPS)".	Closes D2 gap G1; D0's I4 row was wrong.	Section 3.6
DC-04	`budget` is USD per 1,000 requests everywhere.	Closes D2 gap G5; D0's I1 said "USD".	Sections 3.3–3.4
DC-05	The essay constraints are CON-01–CON-04 (formerly D2's NFR-01–NFR-04).	Requirements Document v1.0 change log; NFR-01–04 are retired.	Section 5

2. Architecture Summary

Model Recommend is a **client-server** system with a short **pipeline** inside the server (D0 Part 6). The structure is drawn in the D0 block diagram and D0 data-flow diagram; the run lifecycle is Fig D2-3 and the main sequence is Fig D2-4.

Major components.

- **Recommendation Client:** a React single-page application with six screens (S1–S6). It holds no persistent data; the access token stays in memory.
- **Recommendation Server:** one TypeScript service on Node.js. It contains the API layer (E1–E12), authentication, the run service, prompt sanitization, A1 parallel dispatch with per-vendor limiters, the LLM adapter (`callVendor`), A2 scoring and ranking, review and selection services, the notification dispatcher, configuration and logging.
- **Report Store:** a managed PostgreSQL database holding every record (Fig D2-1).
- **External systems:** the vendor model APIs (OpenAI, Anthropic), an OIDC identity provider, an email provider and an ops webhook (Slack).

Deployment model. The Client is a static bundle served from a static host. The Server is a single Docker image on a managed container platform, scaled by adding instances; the notification dispatcher is safe to run on several instances

because it locks rows with `FOR UPDATE SKIP LOCKED`. The database is managed PostgreSQL reached over TLS. Secrets (vendor keys, database URL, webhook URLs) come from the platform's secret store; all other settings come from the versioned `config/server.yaml`.

Data flow. The product manager fills in the brief on S1 (I1). The Client posts it to E1 (I2). The Server authenticates the token against the identity provider's keys (I5), validates the brief, reads the client company's approved vendor list from the Report Store (I4), creates a Run in `queued` and returns `202`. In the same process, A1 sanitizes the sample prompts and sends them through `callVendor` to every approved vendor's models (I3), writing each call to the usage log. A2 scores and ranks the results and `storeReport` writes the report in one transaction, together with any notifications. The report is released directly, or held in `pending_review` for a solutions engineer. The Client polls E5 every 2 seconds and shows the shortlist on S2. The dispatcher sends queued notifications to email, in-app and the ops webhook (I6).

3. Interfaces and Data Formats

3.1 Boundaries

ID	Boundary	Protocol	Format	Section
I1	Actor ↔ Recommendation Client	Browser UI	HTML forms and views	3.2
I2	Recommendation Client ↔ Recommendation Server	REST over HTTPS	JSON	3.3, 3.4
I3	Recommendation Server ↔ vendor model APIs	HTTPS via vendor SDKs	JSON	3.5
I4	Recommendation Server ↔ Report Store	PostgreSQL wire protocol over TLS	SQL tables	3.6
I5	Recommendation Server ↔ identity provider	OIDC: JWKS over HTTPS	JWT (RS256)	3.7
I6	Recommendation Server → email provider and ops webhook	HTTPS	JSON	3.7
I7	Configuration → Recommendation Server	File and environment, read at startup	YAML and secrets	3.7

3.2 I1: Actor ↔ Recommendation Client

Screen	Role	Inputs	Outputs	Error behavior
S1 Brief form	pm	budget (USD per 1,000 requests), latency ceiling (ms), task description (character counter)	Run started	Field checked in the browser before sending; server 400 shown inline on the named field; 409 shows "Ask your IT reviewer to approve vendors"; 5xx or network shows a banner with Retry
S2 Run progress and shortlist	pm	Cancel, Select	Status chips, ranked entries with fit score and reasoning, warnings, "not saved" badge	Polling backs off on network errors up to 120 s, then Retry resumes the same run
S3 Report history	pm	Page, open report	Report list and report detail	Empty state; 404 shows "report not found"
S4 Vendor approval	reviewer	Vendor checkboxes, optional expiry	Saved list with approval time	"at least 1 vendor required"; 403 shows "reviewer access required"
S5 Review queue and review	solutions_engineer	Notes, new ranks, Release	Updated report	400 too many reorders shown inline; 409 "already released by someone else"
S6 Notifications	all	Mark read	Notification list	Silent retry on next page load

3.3 I2: Endpoints

All endpoints require `Authorization: Bearer <JWT>`; `client_id` and role come from the token, never the body. Bodies are JSON, timestamps ISO-8601 UTC, money USD, latency milliseconds. Every error body is `{"error": "<code>", "message": "<text>", "field": "<name, optional>"}`. The prefix `/v1` is the version; a breaking change ships as `/v2`.

ID	Method and path	Role	Input	Success	Errors
E1	`POST /v1/recommendations`	pm	Brief (3.4) + header `Idempotency-Key` (uuid)	`202 {run_id, status: "queued"}`; same key within 24 h → same run	400 `invalid_brief` + field; 401; 409 `no_approved_vendors`; 409 `run_in_progress` (> 3 active); 429 `rate_limited` + Retry-After; 503 `store_unavailable`, `auth_unavailable`

ID	Method and path	Role	Input	Success	Errors
E2	GET /v1/reports/{report_id}	pm, solutions_engineer	Path uuid	200 Report (3.4) plus brief	400 invalid_id; 401; 404 report_not_found (also for another company's report)
E3	GET /v1/reports	pm	limit 1–100 (default 20), cursor	200 {reports: [{report_id, generated_at, budget, latency_ms, top_model}], next_cursor}	400 invalid_limit, invalid_cursor; 401
E4	PUT /v1/approved-vendors	reviewer	{vendor_ids: uuid[1..50], expires_at?: ≤ 90 days ahead}	200 {vendor_ids, approved_at, expires_at}; replaces the list	400 at_least_one_vendor_required, invalid_expiry; 401; 403 reviewer_access_required; 404 unknown_vendor
E5	GET /v1/runs/{run_id}	pm, solutions_engineer	Path uuid	200 {run_id, status, warnings[], error_code?, report?}	400 invalid_id; 401; 404 run_not_found
E6	POST /v1/runs/{run_id}/cancel	pm	Path uuid	200 {status: "cancelled"}	404; 409 run_not_active
E7	GET /v1/vendors	reviewer	none	200 [{vendor_id, name, retention_policy_url, models: [name]}]	401; 403
E8	POST /v1/reports/{report_id}/review	solutions_engineer	{annotations: [{entry_id, note ≤ 500, new_rank?}], release: bool}	200 {review_status, entries}	400 too_many_reorders, invalid_rank; 403; 404; 409 already_released
E9	POST /v1/reports/{report_id}/selection	pm	{entry_id}	200 {selected_entry_id, selected_at}	400 entry_not_selectable; 404; 409 report_not_released
E10	GET /v1/review-queue	solutions_engineer	limit, cursor	200 {reports, next_cursor}	401; 403
E11	GET /v1/approved-vendors	reviewer, pm	none	200 {vendor_ids, approved_at, expires_at}	401; 404 no_approved_vendors
E12	GET /v1/notifications	all	limit, cursor, unread_only	200 {notifications: [{notification_id, type, payload, created_at, read}], next_cursor}	401

GET /health (no authentication) returns 200 {status: "ok"} for the platform's health check.

3.4 I2: Message schemas

Brief (E1 request), JSON Schema:

```
{
  "type": "object",
  "required": ["budget", "latency_ms", "task_description"],
  "additionalProperties": false,
  "properties": {
    "budget": {"type": "number", "minimum": 0.01, "maximum": 10000, "description": "USD per 1,000 requests"},
    "latency_ms": {"type": "integer", "minimum": 100, "maximum": 60000},
    "task_description": {"type": "string", "minLength": 10, "maxLength": 2000}
  }
}
```

Report (inside E5 and E2 responses): {report_id: uuid, generated_at: string, save_status: "saved" | "not_saved", review_status: "auto_released" | "pending_review" | "released", models: Entry[], warnings: string[]}, where Entry = {entry_id: uuid, rank: integer | null, original_rank: integer | null, name: string, vendor: string, fit_score: integer 0–100 | null, cost_usd_per_1k: number | null, median_latency_ms: integer | null, status: "ok" | "delayed" | "unavailable", within_limits: boolean | null, reasoning: string, notes: string[]}. Warnings include fewer_than_3_models and

deadline_reached. Run status is one of queued, dispatching, scoring, saving, pending_review, released, released_unsaved, selected, failed, cancelled (Fig D2-3).

3.5 I3: Server ↔ vendor model APIs (LLM adapter)

Item	Specification
Signature	callVendor(model, prompt, {max_tokens, signal}) → Promise<VendorResult>
Request to vendor	{model: api_model_name, messages: [{role: "user", content: <sanitized prompt>}], max_tokens, temperature: 0} over HTTPS with the vendor's API key, 20 s timeout
Output	VendorResult = {completion_text, input_tokens, output_tokens, latency_ms, cost_usd, rate_limit_remaining_pct | null}; cost_usd = input_tokens × price_in_per_mtok / 10^6 + output_tokens × price_out_per_mtok / 10^6
Errors	429 → VendorRateLimited(retry_after_s) (A1 counts consecutive 429s; 2 → delayed, REQ-06); timeout → VendorTimeout (retried once); 5xx or network → VendorError (retried once); 401/403 → VendorAuthError (not retried, vendor unavailable, operator alerted); empty text or missing usage → sample failed, never scored as zero
Concurrency	At most 4 calls in flight per vendor, shared across all runs; dispatch deadline 80 s (A1)

3.6 I4: Server ↔ Report Store (database schema)

PostgreSQL, reached over TLS. Migrations are versioned with node-pg-migrate. Full diagram: Fig D2-1.

Table	Key columns	Constraints and indexes
clients	client_id PK, name, requires_se_review (bool, default false)	—
app_users	user_id PK (identity-provider subject), client_id FK (null for staff), role enum (pm, reviewer, solutions_engineer, operator), email	—
vendors	vendor_id PK, name, retention_policy_url	retention_policy_url not null (REQ-07)
models	model_id PK, vendor_id FK, api_model_name, price_in_per_mtok, price_out_per_mtok, is_active	index (vendor_id); soft delete only
vendor_approvals	PK (client_id, vendor_id), approved_at, expires_at null, reminder_sent_at null	partial index on expires_at where not null
runs	run_id PK, client_id FK, status enum, brief jsonb, idempotency_key, error_code, created_at, finished_at	unique (client_id, idempotency_key); index (client_id, created_at DESC)
vendor_calls	call_id PK, run_id FK, model_id FK, kind (benchmark, grade), http_status, outcome, latency_ms, input_tokens, output_tokens, cost_usd, started_at	index (run_id)
reports	report_id PK, client_id FK, run_id FK unique, review_status, selected_entry_id FK null, scoring_version, generated_at	index (client_id, generated_at DESC); partial index on review_status = 'pending_review'
report_entries	entry_id PK, report_id FK, model_id FK, rank null, original_rank, fit_score 0–100 null, cost_usd_per_1k, median_latency_ms, within_limits, status, reasoning	unique (report_id, rank)
report_annotations	annotation_id PK, report_id FK, entry_id FK, author_id FK, note, new_rank null, created_at	index (report_id)
notifications	notification_id PK, recipient_id FK null, type, channel, payload jsonb, state (pending, sent, failed), attempts, next_attempt_at, dedupe_key, read_at	unique dedupe_key; partial index (state, next_attempt_at) where pending

Repository methods. storeReport, getReport, listReports, getApprovedModels, putApprovals (D1 Part 5); createRun, transitionRun, logVendorCall, saveReview, selectEntry, enqueueNotification, nextDueNotifications (D2 2.3). **Error behavior:** WriteFailed on storeReport is retried once, then the run ends released_unsaved with the report held in memory for 1 hour (NFR-15); transitionRun uses UPDATE … WHERE status = <expected> and returns Conflict when another actor won (EC-17); if approvals cannot be read at E1 the request fails closed with 503 store_unavailable (REQ-04); logVendorCall batches every 1 s and never fails a run.

3.7 I5–I7: Identity provider, notification channels, configuration

ID	Inputs	Outputs	Format	Error behavior
I5	Bearer token on each request; the provider's JWKS (cached 10 min)	`Principal {user_id, client_id, role}`	JWT checked for signature, `iss`, `aud`, `exp` (30 s skew)	No or bad token → 401; unknown user → 401 `unknown_user`; wrong role → 403; JWKS unreachable → cached keys for 10 min, then 503 `auth_unavailable`
I6	Due notification rows (every 10 s)	Email via the provider's HTTPS API; ops webhook post	Webhook payload: `{"type": "rate_limit_warning", "vendor": "...", "remaining_pct": 8, "run_id": "...", "at": "..."}`	Channel error → retry up to 5 times with 30 s × 2^n backoff, then `failed` and logged (NFR-16); no email address → in-app only
I7	`config/server.yaml` and platform secrets	Validated `Config` object: `scoring {version, w_q, w_c, w_l}`, `dispatch {prompts_per_model, concurrency_per_vendor, deadline_ms, call_timeout_ms}`, `alerts {rate_limit_remaining_pct, dedupe_minutes}`, `review {auto_release_hours}`, `client_quota {max_active_runs, runs_per_hour}`, `idp {issuer, audience}`	YAML validated by a zod schema	Server refuses to start on `ConfigInvalid` (weights not summing to 1, `deadline_ms` ≥ 90,000); missing vendor key disables that vendor and alerts the operator

4. Technology Stack

Layer	Choice	Justification
Front end	React with Vite, TypeScript	Team's working stack; a small static bundle meets NFR-06 and costs almost nothing to host (CON-01, CON-05).
Back-end runtime	Node.js, TypeScript	A1 is I/O-bound vendor waiting, which the event loop handles without threads (NFR-05); one language for client and server suits a two-person team (CON-05).
HTTP framework	Fastify 5	Low overhead and schema hooks for E1–E12 (NFR-06, REQ-03).
Validation	zod 4	One schema gives runtime validation with field-level errors (REQ-03, NFR-13) and validates configuration at startup (I7).
Concurrency and retry	p-limit 7, p-retry 8	Per-vendor limiters and the retry-once rule (REQ-06, NFR-14, NFR-07).
Vendor clients	Official `openai` and `@anthropic-ai/sdk`	Typed 429 errors and token usage (REQ-06, REQ-11); vendor-maintained, reached under each vendor's own terms (CON-04).
Authentication	External OIDC identity provider; jose 6	No passwords stored; standard JWT checks (NFR-08, CON-02). Provider choice is OQ-04.
Database	Managed PostgreSQL; Kysely 0.29 with pg 8; node-pg-migrate 9	Relational joins, unique (`report_id, rank`) and partial indexes (REQ-01, REQ-09, NFR-06); TLS connections (NFR-10).
Notifications	Outbox table plus in-process dispatcher; email provider API; Slack webhook	Notifications survive channel outages (REQ-10, REQ-15, NFR-16) without a broker (CON-05).
Logging	pino 10 with `redact`	JSON logs with prompt, completion and token paths removed (NFR-09, CON-02).
Testing	vitest; Playwright for end-to-end; mock vendor server	Verifies REQ-06 and NFR-14 without paying for real 429s (CON-01).
Hosting	Docker image on a managed container platform; static host for the Client	Usage-based cost (CON-01); no servers for two people to operate (CON-05). Platform to be confirmed with Concentrate AI (AS-01).
CI	Git hosting CI with secret scanning	Tests on every push; 0 secrets in the repository (NFR-10).

5. Known Constraints

Each constraint from the Requirements Document, Section 8, restated with what it rules out and what it forces.

ID	Constraint	Rules out	Forces
CON-01	Economic: metered vendor spend, sampled evaluation	Exhaustive testing of every model on every workload; self-hosted evaluation models (W-01); a paid message broker	Sampled prompts (5 per model); per-client run caps and idempotency keys (NFR-07, REQ-13); cancel aborts calls; cost logged per call (REQ-11)

ID	Constraint	Rules out	Forces
CON-02	Security: sensitive customer text	Sending raw task text to vendors; plaintext prompt logs; unapproved vendors, including for grading; storing passwords; tokens in browser storage	Sanitizer before every I3 call (NFR-09); grader chosen from approved vendors; fail closed when approvals cannot be read (REQ-04); log redaction; OIDC sign-in (NFR-08)
CON-03	Social: explain trade-offs	A learned or opaque ranker (W-04); scores without numbers behind them	Weighted sum with versioned weights (A2); a reasoning text built from each model's own numbers (REQ-02); `within_limits` flags for near-misses
CON-04	Legal: vendor terms on benchmarking	Public leaderboards or published head-to-head claims (W-02); calls that bypass vendor APIs	Reports visible only to the requesting client company (404 across companies, NFR-08); reasoning states measurements, not "better than" claims; terms review OQ-06 before launch
CON-05	Project: two people, ends April 2027	Microservices, multiple deployed services, a broker, multi-region (D0 D-03, W-06, W-09)	One server image, managed services, in-process concurrency, reuse of mature libraries (D1 Part 4)

6. Traceability Matrix

Design Component names the D0 component and the D1 or D2 element. Status: **Designed** (complete in D0–D2), **In progress** (designed, with an open question that may change it), **Planned** (not yet designed). Test cases TC-01–TC-29 are reserved here and will be written in the Week 10 test plan (Assignment 10). No row is orphaned.

Req ID	Description	Design Component (D0/D1/D2)	Status	Test Case
REQ-01	Ranked shortlist with 0–100 fit scores	Recommendation Server: A1 dispatch (D1), A2 ranking (D1), E1 + E5 (D2 2.2); Report Store: `reports`, `report_entries` (Fig D2-1)	In progress (OQ-01, OQ-07)	TC-01
REQ-02	Reasoning text for every entry	Recommendation Server: A2 reasoning template (D1); `report_entries.reasoning`; Client S2	Designed	TC-02
REQ-03	Invalid brief → 400 naming the field, no dispatch	Recommendation Server: `validateBrief` (zod), E1 (D1 Part 5); Client S1; EC-06, EC-07 (D2)	Designed	TC-03
REQ-04	Prompts only to approved, active vendors; fail closed	Recommendation Server: `getApprovedModels`, A1 grader selection (D1); E1 409/503; Report Store `vendor_approvals`; EC-22, EC-23 (D2)	Designed	TC-04
REQ-05	Reviewer saves 1–50 vendors; 0 rejected	Recommendation Server: E4 (D1), `putApprovals`; Client S4 (D2 2.1); `vendor_approvals.approved_at`	Designed	TC-05
REQ-06	Two 429s → delayed within 5 s, pause, others continue	Recommendation Server: A1 `VendorLimiter.on429` (D1); `callVendor` errors (D2 2.4); EC-11	Designed	TC-06
REQ-07	Vendor list with retention-policy links	Recommendation Server: E7 (D2); Report Store `vendors.retention_policy_url`; Client S4	Designed	TC-07
REQ-08	Product manager selects one ok entry	Recommendation Server: E9, `selectEntry` (D2 2.2–2.3); `reports.selected_entry_id`; EC-20	Designed	TC-08
REQ-09	Solutions-engineer review, ≤ 2 moves	Recommendation Server: E8, E10, `pending_review` state (Fig D2-3); `report_annotations`, `original_rank`; Client S5; EC-17, EC-18, EC-25	Designed	TC-09
REQ-10	Operator rate-limit warning	Recommendation Server: A1 + `rate_limit_remaining_pct` (D2 2.4); Notifications glue `rate_limit_warning` (D2 2.6, Fig D2-5); EC-37	In progress (OQ-03)	TC-10
REQ-11	Every vendor call in the usage log	Logging glue: `logVendorCall` (D2 2.8); Report Store `vendor_calls`; EC-44, EC-46	Designed	TC-11
REQ-12	Report history; other companies → 404	Recommendation Server: E2, E3 (D1); Report Store index `(client_id, generated_at)`; Client S3; EC-24, EC-35	Designed	TC-12
REQ-13	Cancel within 5 s; idempotent resubmission	Recommendation Server: E6, `Idempotency-Key` on E1 (D2 2.2); `runs` unique key; Client S2; EC-02, EC-09	Designed	TC-13
REQ-14	90-day expiry and 7-day reminder	Report Store `vendor_approvals.expires_at`, `reminder_sent_at` (D1, D2); `ApprovalReminderJob`; EC-40	Designed	TC-14
REQ-15	Product managers notified when list is active	Notifications glue `vendor_list_active` (D2 2.6); outbox in E4 transaction	Designed	TC-15
REQ-16	Auto-release after 24 h	Recommendation Server `ReviewTimeoutJob` (D2 2.2, Fig D2-3); `review_overdue` notification; EC-19	Designed	TC-16

Req ID	Description	Design Component (D0/D1/D2)	Status	Test Case
NFR-05	95% of runs terminal within 90 s	Recommendation Server: A1 80 s deadline and limiters (D1); run lifecycle (Fig D2-3); config check EC-43	Designed	TC-17
NFR-06	Read endpoints p95 ≤ 500 ms	Report Store indexes (D1 Part 2, D2 2.3); E2, E3, E5, E7, E11, E12	Designed	TC-18
NFR-07	Call cap per run; 3 active and 20 per hour per company	Configuration `client_quota`, `dispatch` (D2 2.7); E1 409/429; A1 deduplication (D1); EC-08	Designed	TC-19
NFR-08	401/403/404 matrix	Authentication glue `authenticate`, `requireRole` (D2 2.5); `app_users`; EC-31–EC-35	In progress (OQ-04)	TC-20
NFR-09	Redaction before vendors; no secrets in logs	Recommendation Server `PromptSanitizer` (D1 Part 4); Logging glue pino `redact` (D2 2.8); EC-16, EC-45	In progress (OQ-05)	TC-21
NFR-10	TLS, in-memory token, 0 secrets in repo	Client token handling (D2 2.1); Configuration secrets (D2 2.7); hosting (D1 Part 6); CI secret scan	Designed	TC-22
NFR-11	4 of 5 PMs finish a brief in 3 min	Recommendation Client S1, S2 and state machine (Fig D2-2)	Designed	TC-23
NFR-12	4 of 5 reviewers approve in 2 min	Recommendation Client S4 (D2 2.1)	Designed	TC-24
NFR-13	Plain-language errors	Recommendation Client screen error column (D2 2.1); error envelope (Section 3.3)	Designed	TC-25
NFR-14	One failing vendor does not fail the run	A1 per-vendor isolation (D1); `callVendor` error mapping (D2 2.4); EC-10, EC-11, EC-26, EC-27	Designed	TC-26
NFR-15	No silent loss on write failure or restart	Report Store `storeReport` retry (D0 I4, D1); `RunRecovery.onStartup` (D2 2.2); EC-15, EC-21	Designed	TC-27
NFR-16	Notification retry and deduplication	Notifications glue dispatcher, dedupe_key (D2 2.6, Fig D2-5); EC-36–EC-38	Designed	TC-28
CON-01	Economic: metered spend	`vendor_calls.cost_usd` (D2), `client_quota` (D2 2.7), E6 cancel, Idempotency-Key, sampled prompts in A1 (D1)	Designed	TC-11, TC-13, TC-19
CON-02	Security: sensitive text	`PromptSanitizer`, approved-vendor check, log redaction, OIDC (D1, D2 2.5, 2.8)	In progress (OQ-05)	TC-04, TC-20, TC-21
CON-03	Social: explainable	A2 weighted sum and reasoning template (D1); `within_limits`; review notes (D2)	Designed	TC-02
CON-04	Legal: vendor terms	Report scoped to one client company (D1 E2, EC-35); reasoning wording; vendor SDKs only (D1 Part 4)	In progress (OQ-06)	TC-12, TC-29 (inspection of reasoning wording)
CON-05	Project: two people, April 2027	Client-server pattern and single server (D0 Part 6, D-03); managed hosting (D1 Part 6)	Designed	Checked at milestone reviews M1–M7 (Plan.md), not by a test case

7. Advisor Review

Status: requested, not yet held. This assignment is due on 2026-10-30, one day after both drafts were completed, so the three-day lead time for advisor review was not possible. Both drafts and this matrix are being sent to Jillian Aurisano and Todd Liebermann on 2026-10-29 with a request for a meeting in the week of 2026-11-02.

Agenda for the meeting.

1. Walk through the traceability matrix (Section 6), especially the five *In progress* rows.
2. Decide OQ-01 (fewer than 3 models), OQ-02 (two 429s) and OQ-07 (scoring weights).
3. Todd Liebermann: OQ-03 (alert threshold), OQ-04 (identity provider), OQ-06 (vendor terms).
4. Check that the Must set (10 of 28) is a realistic MVP for M4.

Record (to be completed after the meeting).

Field	Entry
Date and attendees	*Pending*
What was discussed	*Pending*
Changes it produced	*Pending.* Each change will be logged in both change logs as version 1.1, and the matrix updated.